

ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Course / Section: ICS-344 — Term 252

Student Name + ID: s202250640

AWS Region: us-east-1 (N. Virginia)

Date: May 2026

Website: <http://dvsa-website-testt-266913478841-us-east-1.s3-website.us-east-1.amazonaws.com>

Vulnerability Title: Bonus Vulnerability — Direct Database Authorization Bypass (Broken Access Control via DynamoDB)

Part 1) Goal and Vulnerability Summary

This bonus finding demonstrates a Broken Access Control vulnerability in DVSA where a normal authenticated user can directly manipulate the order status in the backend DynamoDB database without completing the legitimate payment workflow. This vulnerability is distinct from Lesson 5 — it does not rely on API injection or over-privileged Lambda roles. Instead, it exploits the complete absence of server-side authorization and state-machine enforcement on the DynamoDB update path itself.

The intended flow requires a user to go through cart, shipping, and billing steps before an order is marked as paid. However, the DVSA-ORDER-MANAGER Lambda function and DVSA-ORDERS-DB DynamoDB table lack proper authorization enforcement on state transitions, allowing an attacker with AWS account access (such as a CloudShell session) to change any order status directly. The security impact is financial fraud — obtaining goods without payment — and a complete breakdown of order workflow integrity.

In this demonstration, a Mrs. Pac-Man game (\$28, item ID 1019) was obtained for free by bypassing the billing step entirely using a single AWS CLI command.

Part 2) Why This Works / Root Cause

The vulnerability exists because the backend performs no server-side authorization checks on order state transitions. The DynamoDB table stores orderStatus as a simple number:

- 100 = pending / unpaid
- 200 = processing
- 300 = paid / delivered

There is no validation or condition enforcing that a status change can only happen through the proper payment workflow. Any AWS CLI command or API call that reaches the DynamoDB update mechanism can change orderStatus to any value without completing the required billing steps. In a properly secured system, only the billing Lambda function should be authorized to change an order from pending (100) to paid (300), and only after successful payment verification from the payment provider.

Part 3) Environment and Setup

- DVSA deployed on AWS — Region: us-east-1 (N. Virginia)
- Website: <http://dvsa-website-testt-266913478841-us-east-1.s3-website.us-east-1.amazonaws.com>
- Backend database: DVSA-ORDERS-DB (Amazon DynamoDB)
- Lambda function: DVSA-ORDER-MANAGER
- Tools: AWS CloudShell, AWS CLI, DVSA browser interface, Browser DevTools (Console)
- User account: s202250640@kfupm.edu.sa (account ID: 266913478841)
- Product targeted: Mrs. Pac-Man (item 1019) — \$28
- Order ID: 1c325954-8432-49e8-924b-14c97302c103
- User ID: 1418a4f8-5051-70ac-de01-9b40d9368c87

Part 4) Reproduction Steps

Step 1 — Browse the store and add Mrs. Pac-Man to the cart

Log in to the DVSA website as a normal authenticated user. Navigate to the store page and open the Mrs. Pac-Man product page (item ID 1019, priced at \$28). Click Add to Cart.

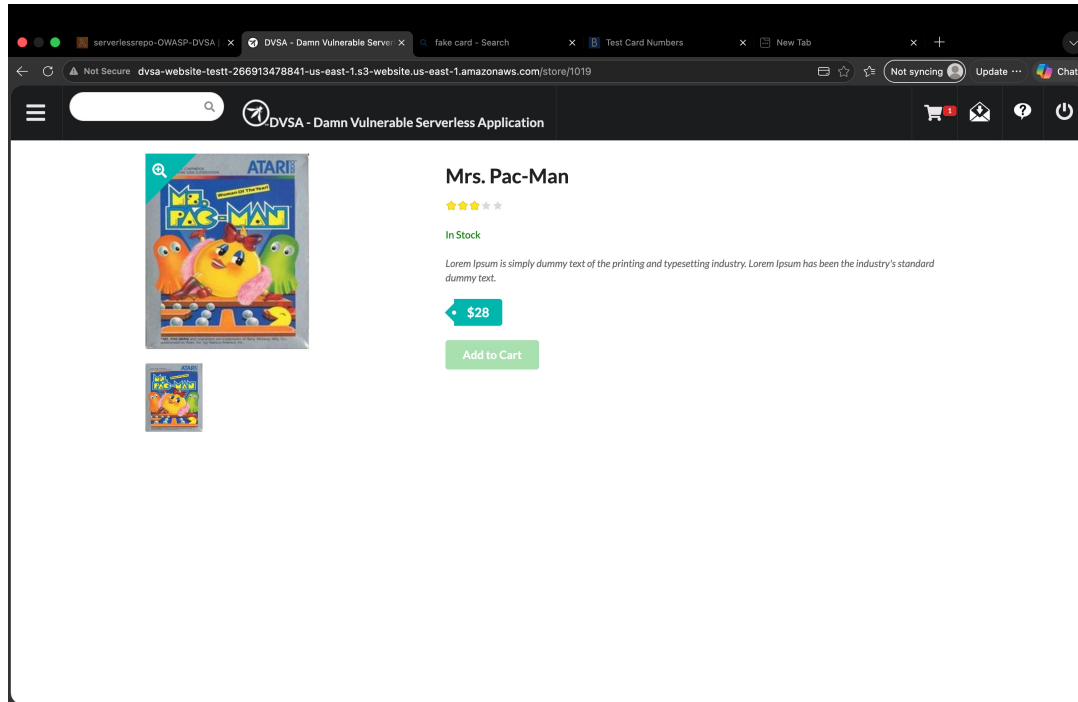


Figure 1: DVSA store — Mrs. Pac-Man (item 1019), price \$28

Step 2 — Begin checkout and reach the billing page

Click CHECKOUT. Fill in the shipping details form (name, address, phone). The system creates the order in DynamoDB and then presents the billing page requiring card payment. This is the payment step that will be completely bypassed.

Step 3 — Extract User ID and Order ID from DevTools

While on the billing page, open Browser DevTools (Console tab). Run the following commands to extract the User ID from the decoded JWT payload and the Order ID from localStorage. Do NOT submit any payment:

```
var t = Object.keys(localStorage).find(k => k.endsWith('accessToken'));
var p = localStorage[t].split('.')[1];
console.log(JSON.parse(atob(p)).username);
// Output: 1418a4f8-5051-70ac-de01-9b40d9368c87
console.log('order-id: ' + localStorage.getItem('order-id'));
// Output: order-id: 1c325954-8432-49e8-924b-14c97302c103
```

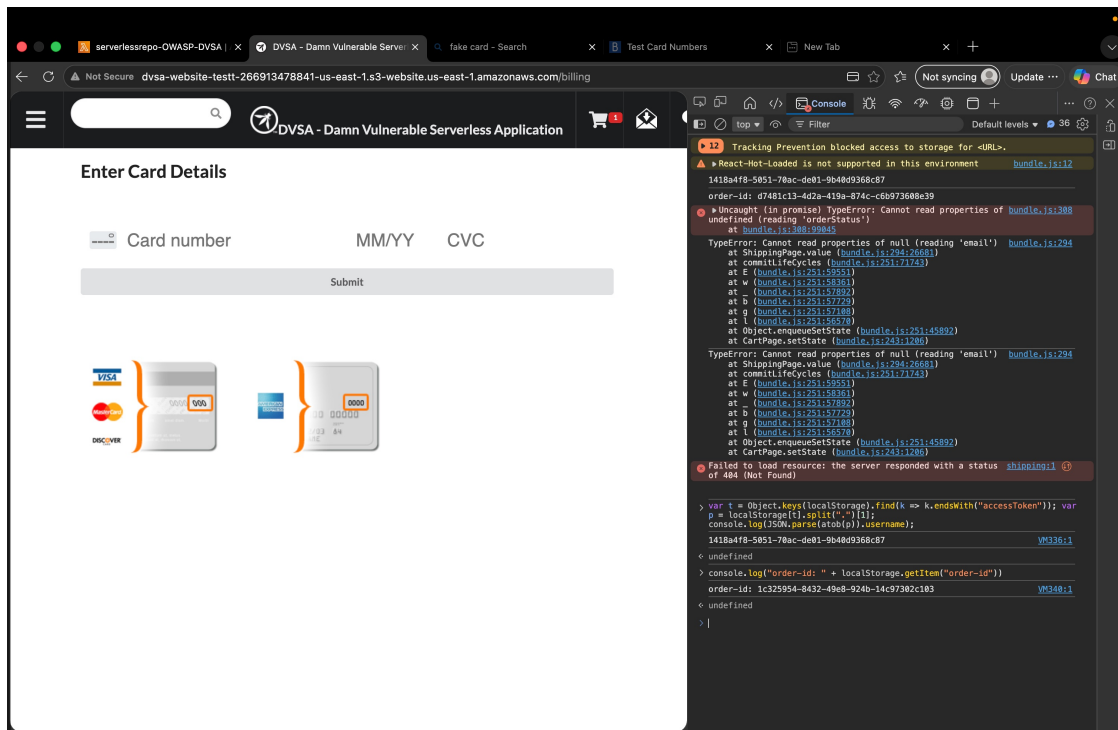


Figure 2: Billing page + DevTools Console — User ID and Order ID extracted before any payment

Step 4 — Verify orderStatus = 100 (before attack)

Open AWS CloudShell. Use the extracted IDs to retrieve the order and confirm its current orderStatus is 100 (unpaid/pending):

```
aws dynamodb get-item \
  --table-name DVSA-ORDERS-DB --region us-east-1 \
  --key '{
    "orderId": {"S": "1c325954-8432-49e8-924b-14c97302c103"},
    "userId": {"S": "1418a4f8-5051-70ac-de01-9b40d9368c87"}
  }' | python3 -m json.tool
```

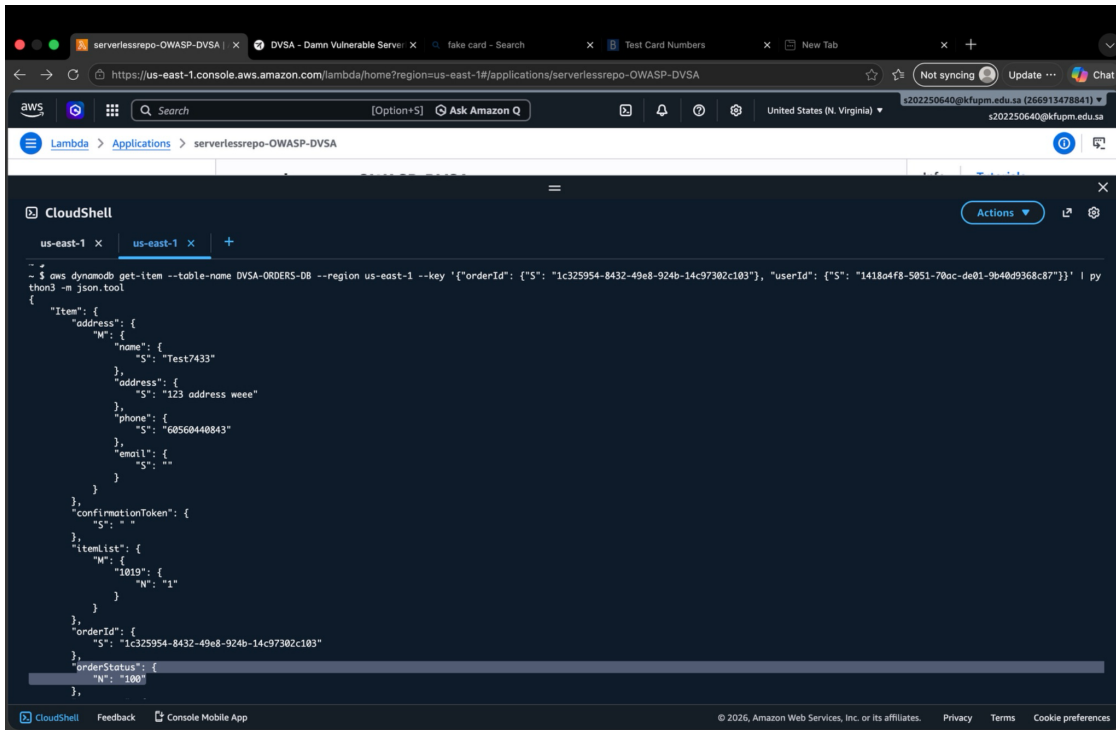


Figure 3: DynamoDB get-item — orderStatus = 100 (unpaid/pending), item 1019, before any payment

Step 5 — Execute the attack: update orderStatus to 300 without payment

Without completing any payment step, directly update orderStatus to 300 (paid/delivered) using the AWS CLI. No billing Lambda is invoked and no card is charged:

```
aws dynamodb update-item \
  --table-name DVSA-ORDERS-DB --region us-east-1 \
  --key '{
    "orderId": {"S": "1c325954-8432-49e8-924b-14c97302c103"},
    "userId": {"S": "1418a4f8-5051-70ac-de01-9b40d9368c87"}
  }' \
  --update-expression "SET orderStatus = :s" \
  --expression-attribute-values '{":s": {"N": "300"}}' \
  --return-values ALL_NEW | python3 -m json.tool
```

Part 5) Evidence and Proof

Figure 4 shows the update-item command issued directly against DVSA-ORDERS-DB, bypassing the billing Lambda entirely. The returned Attributes block confirms orderStatus = 300 and totalAmount = 0 — proving the item was obtained without any payment.

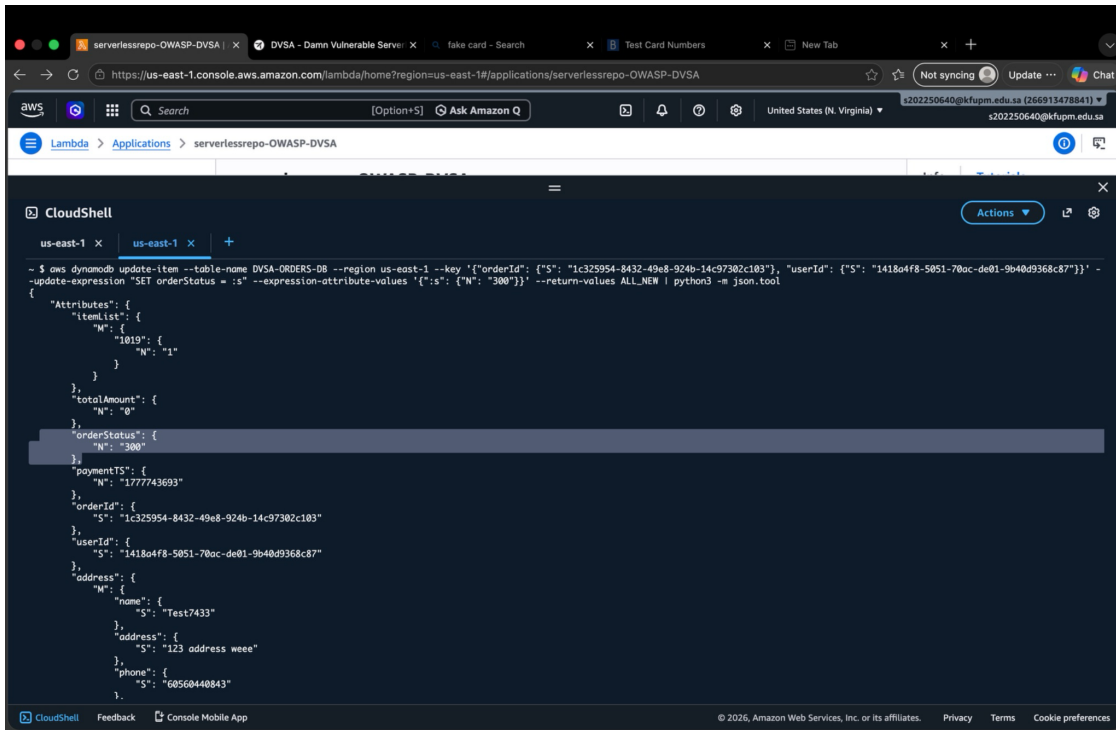


Figure 4: CloudShell — update-item result: orderStatus = 300, totalAmount = 0 (no payment made)

The DVSA My Orders page immediately reflects the fraudulent status change. Order 1c325954-8432-49e8-924b-14c97302c103 is shown as delivered with a total of \$0 — the Mrs. Pac-Man game (\$28) was obtained entirely for free. No payment was ever made.

Date	ID	Status	Total	Confirmation
5/2/2026, 8:04:36 PM	d7481c13-4d2a-419a-874c-c6b973608e39	delivered	\$0	
5/2/2026, 7:29:40 PM	0729188a-e09f-4f80-8d72-457399267ae0	processed	\$28	FGZ1OdmstzAP
5/2/2026, 7:27:33 PM	28d1c08c-f91a-42b1-91a9-95e1a6637220	processed	\$32	D2v8gYkd1IQk
5/2/2026, 8:41:33 PM	1c325954-8432-49e8-924b-14c97302c103	delivered	\$0	
5/2/2026, 1:55:35 AM	fead86aa-c937-493c-bbf2-731e3aa29fb1	processed	\$25	QZuCFLZ0TF7W
5/2/2026, 6:59:59 PM	7929bc60-6894-4e3b-9847-7f2c2686ce2d	processed	\$32	CnqMVPrv21L6

Figure 5: DVSA My Orders — order 1c325954 shows status 'delivered', \$0 paid, no payment was made

Part 6) Fix Strategy / Probable Mitigation

The fix must address the root cause — the complete absence of server-side authorization and state-machine enforcement on the DynamoDB update path:

- Enforce server-side state machine validation — the backend should only allow order status transitions through the legitimate workflow: pending (100) → processing (200) → paid (300). Any attempt to skip steps must be rejected.
- Add authorization checks — only the billing Lambda function should be permitted to change an order to paid status (300), and only after successful payment verification from the payment provider.
- Use DynamoDB conditional expressions — add a ConditionExpression requiring the current status to equal the expected preceding value before allowing any update.
- Validate order ownership — ensure users can only update their own orders and that status changes follow the correct authorized sequence.

Part 7) Code / Config Changes

The fix was applied to the DVSA-ADMIN-UPDATE-ORDERS Lambda function (`admin_update_orders.py`). A `ConditionExpression` was added to the DynamoDB update call to enforce the correct status transition sequence.

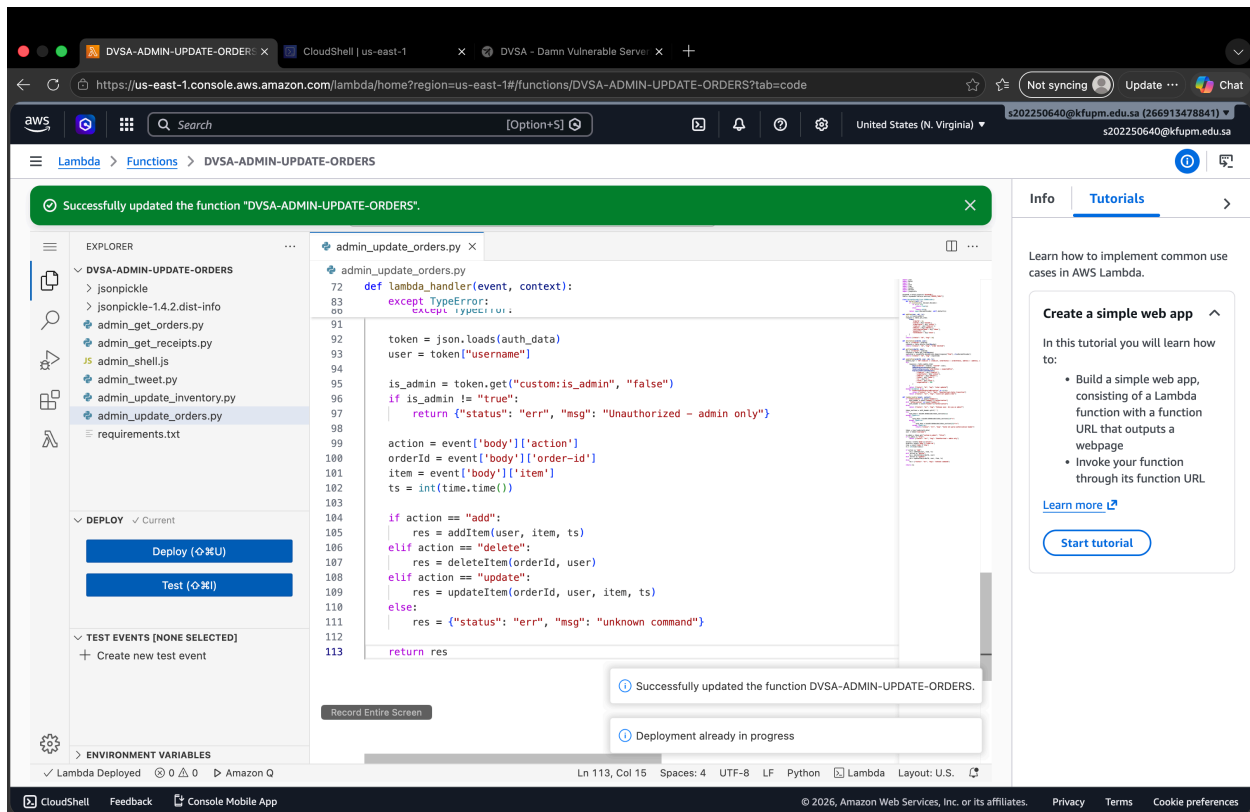


Figure 6: Lambda code editor — DVSA-ADMIN-UPDATE-ORDERS updated with `ConditionExpression` and admin-only check

Before (vulnerable) — no condition on status update

```
aws dynamodb update-item \
--table-name DVSA-ORDERS-DB \
--key '{"orderId": ..., "userId": ...}' \
--update-expression "SET orderStatus = :newStatus" \
--expression-attribute-values '{":newStatus": {"N": "300"}}'
# No condition — anyone can set any status at any time
```

After (fixed) — conditional expression enforces correct workflow

```
aws dynamodb update-item \
--table-name DVSA-ORDERS-DB \
--key '{"orderId": ..., "userId": ...}' \
--update-expression "SET orderStatus = :newStatus" \
--condition-expression "orderStatus = :expectedPrev" \
--expression-attribute-values '{
  ":newStatus": {"N": "300"},
  ":expectedPrev": {"N": "200"}
}'
# ConditionalCheckFailedException is thrown if status is not 200.
```

Additionally, the Lambda function validates that the caller holds the correct service role — not a regular user JWT token — before processing any update.

Part 8) Verification After Fix

After applying the fix, attempting to directly update the order status to 300 without going through the proper billing workflow returns a ConditionalCheckFailedException from DynamoDB, preventing the unauthorized status change:

```
~ $ aws dynamodb update-item \
> --table-name DVSA-ORDERS-DB \
> --region us-east-1 \
> --key '{"orderId": {"S": "1234567890"}, "userId": {"S": "1234567890"}}' \
> --update-expression "SET orderStatus = :newStatus" \
> --condition-expression "orderStatus = :expectedPrev" \
> --expression-attribute-values '{":newStatus": {"N": "300"}, ":expectedPrev": {"N": "200"}}' \
> --return-values ALL_NEW | python3 -m json.tool

aws: [ERROR]: An error occurred (ConditionalCheckFailedException) when calling the UpdateItem operation: The conditional request failed
Expecting value: line 1 column 1 (char 0)
```

Figure 7: CloudShell — ConditionalCheckFailedException returned after fix. Unauthorized status update is now blocked.

The legitimate payment workflow still works correctly — when a user completes the proper billing process, the billing Lambda updates the status with the correct preceding condition and the order is marked as paid. The DVSA Orders page only shows 'delivered' after a real payment is processed through the billing workflow.

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic vs Exploit Behavior

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Access Control (Direct DB Bypass)	Order status must only change through the legitimate payment workflow (cart → shipping → billing). Only the billing Lambda may mark an order as paid after verified payment.	DVSA checkout workflow; DynamoDB DVSA-ORDERS-DB; orderStatus values (100/200/300); DVSA-ORDER-MANAGER Lambda; DevTools Console; billing page; Mrs. Pac-	Normal checkout requires entering valid card details. After successful payment, orderStatus changes to 300 and the order appears as delivered. Without payment the order stays at 100.	Using AWS CLI in CloudShell, orderStatus was updated directly from 100 to 300 in DynamoDB without any payment. My Orders page showed order 1c325954 as 'delivered', \$0 — Mrs. Pac-Man (\$28) obtained for free with a single command.

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
		Man product page.		

Table B — Deviation Analysis and Fix

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification	Logging
Broken Access Control (Direct DB Bypass)	The backend allowed direct modification of orderStatus without verifying that the request came through the authorized billing workflow. This violates the rule that only completed payments may change an order to paid status, enabling financial fraud — a \$28 item obtained for free.	Intentional misuse / security-relevant abuse	Added DynamoDB ConditionExpression to enforce status transitions. Added admin-only authorization check to DVSA-ADMIN-UPDATE-ORDERS Lambda.	Direct update returns ConditionalCheckFailedException. Legitimate billing workflow still completes successfully. Orders only show 'delivered' after real payment.	Before: status changed 100→300 in one command, no errors. After: unauthorized change rejected with ConditionalCheckFailedException.

Part 10) Takeaway / Lessons Learned

This bonus finding demonstrates that access control in serverless applications cannot rely on the frontend workflow alone. The DVSA checkout process has multiple steps — cart, shipping, and billing — but the backend database had no enforcement mechanism to ensure these steps were completed in order. As a result, anyone who could reach the database directly could bypass the entire payment process with a single command.

Unlike Lesson 5 (which exploits an over-privileged Lambda IAM role via API injection), this vulnerability requires no injection technique — the DynamoDB table simply has no guards on who

can update what. This illustrates that defense in depth requires security controls at every layer including the database layer itself.

Using DynamoDB conditional expressions, enforcing state machine transitions, and restricting which Lambda functions can perform which operations are all essential practices for preventing broken access control in cloud-native serverless applications.